

mdstats 3-D Graphics User Guide

Practical configurable framework, connectivity, trajectory, and density visualization with
mdstats-3d

mdstats user guide

2026-08-11

Contents

1	What <code>mdstats-3d</code> does	1
2	Source-tree and installed commands	2
3	Fast layer selection	2
4	Reproduce the historical LTA hybrid plot	2
5	Use TOML for publication scenes	2
6	Preview the canonical scene before computing it	3
7	Configuration precedence	4
8	View, visibility, and periodic display	4
9	Output protection	5
10	LAMMPS input	5
11	Useful resource/render controls	5
12	Shared scientific dependencies and renderer-neutral output	5
13	What comes next	5
13.1	Robust repeated plotting in 0.20.156a0	6

1 What `mdstats-3d` does

`mdstats-3d` is the universal configurable 3-D scene command introduced in `mdstats` 0.20.148a0; 0.20.149a0 adds the shared scientific dependency DAG, 0.20.150a0 completes renderer-neutral composition/view semantics, and 0.20.156a0 hardens topology reuse, long-trajectory framework preparation, validation, hover, scene-owned cell display, and browser-memory preflight.

The first four independent graphical layer types are:

<code>framework</code>	<code>zeolite/framework topology</code>
<code>connectivity</code>	<code>atomic mean connectivity</code>
<code>trajectory</code>	<code>selected atomic trajectories</code>
<code>density</code>	<code>selected atomic probability density</code>

You may display any combination and may use more than one instance of the same type.

2 Source-tree and installed commands

From an mdstats source checkout:

```
python tools/mdstats-3d.py TRAJECTORY ...
```

After package installation:

```
mdstats-3d TRAJECTORY ...
```

3 Fast layer selection

Framework only:

```
mdstats-3d dump.lammpstrj \  
  --lammps-units metal \  
  --lammps-timestep 0.001 \  
  --lammps-type-map "1=Si,2=Al,3=O,4=Na" \  
  --layer framework
```

Framework plus Na trajectories:

```
mdstats-3d dump.lammpstrj \  
  --lammps-units metal \  
  --lammps-timestep 0.001 \  
  --lammps-type-map "1=Si,2=Al,3=O,4=Na" \  
  --layer framework \  
  --layer trajectory:Na
```

Na-O connectivity plus Na density:

```
mdstats-3d dump.lammpstrj \  
  --lammps-units metal \  
  --lammps-timestep 0.001 \  
  --lammps-type-map "1=Si,2=Al,3=O,4=Na" \  
  --layer connectivity:Na-O \  
  --layer density:Na
```

Give a layer a custom name with @:

```
--layer density:Na@mobile-Na-density
```

4 Reproduce the historical LTA hybrid plot

The former `examples/plot_lta_mixed_alkali_density.py` workflow is now a preset:

```
mdstats-3d dump.lammpstrj \  
  --preset lta-mixed-alkali-density \  
  --lammps-units metal \  
  --lammps-timestep 0.001 \  
  --lammps-type-map "1=Si,2=Al,3=O,4=Na"
```

The preset detects which of Si, Al, O, Li, Na, and K are present and builds framework, atomic connectivity, trajectories, and one density layer per present supported species.

The old example script still works as a compatibility launcher.

5 Use TOML for publication scenes

For a reusable figure, prefer TOML over a long shell command.

Example `na_lta.toml`:

```

[scene]
title = "Na-LTA 300 K"
registration = "framework_registered"
display_cell = "reference"
trajectory_display_mode = "folded"
density_grid_interval = 0.20
density_adaptive_smearing = true
projection = "orthographic"

[input]
format = "lammps-dump"
lammps_units = "metal"
lammps_timestep = 0.001
lammps_type_map = "1=Si,2=Al,3=O,4=Na"

[resources]
max_threads = 8
max_memory = "12GiB"

[output]
path = "na_lta.html"
manifest = "na_lta.scene.json"
browser_profile = "balanced"

[[layer]]
type = "framework"
name = "LTA framework"

[[layer]]
type = "connectivity"
name = "Na-O coordination"
selection = { pairs = ["Na-O"] }
render = { edge_width = 1.6, edge_opacity = 0.45 }

[[layer]]
type = "trajectory"
name = "Na trajectories"
selection = { species = ["Na"] }
render = { line_width = 1.6, opacity = 0.28, show_legend = true }

[[layer]]
type = "density"
name = "Na density"
selection = { species = ["Na"] }
render = { mass_fractions = [0.50, 0.80, 0.95], inner_opacity = 0.22, outer_opacity = 0.04 }

```

Run it with:

```
mdstats-3d dump.lammpstrj --config na_lta.toml
```

Paths written inside the TOML file are resolved relative to the TOML file.

6 Preview the canonical scene before computing it

Use manifest-only mode:

```
mdstats-3d dump.lammpstrj \
  --config na_lta.toml \
  --manifest-only
```

This reads/resolves the source and writes the normalized scene manifest without building topology, connectivity, density, or Plotly geometry.

To print the manifest as well:

```
--print-manifest
```

This is useful for checking exactly which named layers will be present and how selections were normalized.

7 Configuration precedence

The rule is:

```
defaults < preset < TOML < explicit CLI
```

Layer lists are replaced, not merged.

For example, if `na_lta.toml` contains four layers but you run:

```
mdstats-3d dump.lampstrj \
  --config na_lta.toml \
  --layer framework \
  --layer density:Na
```

only those two CLI layers are displayed.

8 View, visibility, and periodic display

View controls are universal and do not change the scientific products in the scene. For example:

```
mdstats-3d dump.lampstrj \
  --config na_lta.toml \
  --camera "[111]" \
  --periodic-images 2x2x1 \
  --visible-layer "LTA framework" \
  --visible-layer "Na density"
```

Useful view options are:

```
--projection orthographic|perspective
--camera [100]|[110]|[111]|isometric
--periodic-images 2x2x1
--cell-mode reference|none
--visible-layer NAME
--show-axes
--background light|dark|transparent
--width N
--height N
```

`--visible-layer` may be repeated. Layers not listed start hidden but remain inside the self-contained HTML and can be toggled from the legend without recomputation. Plotly group-click toggles a complete named layer.

Equivalent TOML is:

```
[scene]
projection = "orthographic"
camera = "[111]"
periodic_images = "2x2x1"
visible_layers = ["LTA framework", "Na density"]
cell_mode = "reference"
```

Periodic replication is display-only. A 2x2x1 scene does not multiply density normalization, trajectory weights, connectivity occupancy, or any scientific evidence.

A layer may also set `priority = -10, 0, 10`, etc. in its `[[layer]]` table to control draw order. Priority is render-only; declaration order breaks ties.

9 Output protection

`mdstats-3d` does not overwrite an existing HTML or scene manifest by default.

To replace them deliberately:

```
--force
```

Normal execution writes both the HTML and the canonical `.scene.json` evidence.

10 LAMMPS input

If the dump has numeric `type` rather than `element`, supply a type map:

```
--lammeps-type-map "1=Si,2=Al,3=O,4=Na"
```

If the dump does not record the unit style or physical time, also supply information such as:

```
--lammeps-units metal  
--lammeps-timestep 0.001
```

or provide a suitable LAMMPS log with `--lammeps-log`.

The CLI intentionally fails instead of guessing these quantities.

11 Useful resource/render controls

```
--max-threads N  
--max-memory 12GiB  
--wall-time-target 1200  
--browser-profile compact|balanced|quality  
--max-browser-faces N
```

The wall-time value is advisory; it is not a hard scientific stop.

12 Shared scientific dependencies and renderer-neutral output

The command surface and layer composition are universal, and raw preparation now exposes separate framework-topology, atomic-connectivity, trajectory, and density product dependencies. Omitted layer families omit their dependency key. Duplicate instances share equal keys, and concurrent requests for one scientific key execute once within a scene context.

The LTA provider still uses the qualified framework-dynamics machinery internally when that scientific owner needs joint registration/density planning. That batching is hidden behind the dependency DAG and does not make the composite scene object the scientific dependency of every layer. As of GFX3D-5, prepared layers also no longer carry the composite scene merely so the renderer can work: each adapter emits renderer-neutral primitives and the common Plotly backend only knows primitive classes.

13 What comes next

The common GFX3D foundation is complete through GFX3D-5. The next visualization extension is **GFX3D-RING1**, which will expose the existing ring scientific authority as an independent registered layer before cage, site, assignment, transition-path, and Markov-network layers are added.

13.1 Robust repeated plotting in 0.20.156a0

Repeated LTA plots now reuse the automatically written topology sidecar only after exact authentication against the parsed trajectory, framework connectivity policy, and framework mapping. If the trajectory geometry, frame selection/stride, mapping, or relevant cutoff policy changes, mdstats rejects the stale sidecar and rebuilds it. Use `--no-topology-cache` only when you explicitly want to force that rebuild on every run.

The cell is now a scene-level view element, so `cell_mode = "reference"` works for framework-only, trajectory-only, connectivity-only, and density-only scenes. Camera vectors and periodic replication counts are strict; invalid/fractional declarations produce an error instead of being rounded or truncated. Large periodic display requests are checked against the browser budget before replicated geometry is allocated.

For long fixed-cell framework trajectories, 0.20.156a0 adds a vectorized projected-geometry path while preserving the established frame-local algorithm as the fallback for variable-cell and periodic-multiedge cases. This is execution-only: changing worker count or taking the fast path does not change the framework scientific identity.